Improving Synchronization and Channel Estimation Robustness for OFDM-Based Integrated Sensing and Communication in GNU Radio

Akshita Muchhal
Project ID: g16, gsoc isac (Individual Project)
Ahmedabad University

M2 Report, October 11, 2026

1 Summary of M2 progress

M1 set up the problem: GNU Radio has OFDM transceiver blocks in gr-digital and OFDM radar blocks in gr-radar, but nothing connects them for a single shared waveform. The M1 timeline reserved M2 for finding a specific, scoped contribution (a bug fix, a test-suite extension or a block modification) and for starting the conversation with maintainers.

For M2, I made a block modification in GNU Radio Companion (GRC) and submitted it upstream. The change adds a Required option to the Parameter block. When a flowgraph author sets it to Required, the generated Python script marks the matching command-line argument as mandatory. Optional parameters keep their configured default value. The pull request is gnuradio/gnuradio#8233 and it references issue #5096.

This contribution is a change to GRC code generation. It is a different layer from the ISAC synchronization problem described in M1, and Section 7 says how the two relate. Its main value for the project so far is that I have now gone through the whole contribution process on the real GNU Radio repository: branch, tests, commit, DCO sign-off, fork, and pull request.

Table 1: Contribution at a glance.

Item	Detail
Project / component	GNU Radio / GNU Radio Companion (GRC)
Contribution type	Feature enhancement (block modification)
Pull request	github.com/gnuradio/gnuradio/pull/8233
Related issue	github.com/gnuradio/gnuradio/issues/5096
Fork and branch	MuchhalAkshita/gnuradio, branch grc-required-parameters
Commit	f7afafee6ccdba2ea8bd281dc6063b2f0744f136, "grc: add required option to parameter block", signed off
Size	4 files changed, 30 insertions, no deletions
Environment	Linux; source tree at ~/development/gnuradio. The exact Ubuntu version was not confirmed.

2 The contribution

2.1 Problem

A GRC Parameter block can become a command-line argument in the generated Python script. Before this change, the generated argument definition used the parameter default, so the argument was always optional:

```
parser.add_argument(
    "--required-count",
    dest="required_count",
    type=int,
    default=10)
```

If a flowgraph needs the user to make an explicit choice, silently falling back to 10 is the wrong behavior. The flowgraph author had no way to tell argparse that the argument must be supplied.

2.2 What the change does

- Adds a Required setting to the Parameter block.
- Generates required=True for required parameters in the generated argparse definition.
- Keeps the configured default for optional parameters.
- Applies the same logic to the no-GUI, Qt GUI and Bokeh GUI Python workflow templates.
- Documents the setting in the block documentation.
- Defaults to Optional so that older flowgraphs keep their behavior.

2.3 Why the change is technically meaningful

The YAML block definition only describes the properties dialog. The Mako workflow templates decide what Python code is written. Adding the field to the YAML alone would change the dialog and nothing else, so the feature has to be implemented in the code generator. That is why the change spans the block definition and three templates.

The change is also deliberately additive: 30 lines added and none removed. The optional path produces the same code as before, which keeps the review small and the risk to existing flowgraphs low.

3 Implementation

3.1 Block definition

A new field was added to grc/blocks/parameter.block.yml after the existing value parameter:

```
-   id: required
    label: Required
    dtype: bool
    default: 'False'
    options: ['True', 'False']
    option_labels: [Optional, Required]
```

The dialog shows the two choices as Optional and Required. The documentation section gained two sentences: "Set Required to True to make the command line argument mandatory. Otherwise, the parameter uses its configured default value when the argument is omitted."

3.2 No-GUI workflow template

grc/workflows/python_nogui/flow_graph_nogui.py.mako reads the new field and picks one of two argument definitions:

```
required_ = param.params['required'].get_value()

% if type_:
% if required_ == 'True':
parser.add_argument(
    ${ ', '.join(switches) }, dest="${param.name}", type=${type_}, required=True,
    help="Set ${param.params['label'].get_evaluated() or param.name}")
% else:
parser.add_argument(
    ${ ', '.join(switches) }, dest="${param.name}", type=${type_}, default=${ default },
    help="Set ${param.params['label'].get_evaluated() or param.name} [default=%(default)r]")
% endif
% endif
```

With Required set, argparse rejects a run that omits the argument. Without it, the template writes the same default-based definition as before, including the default in the help text.

3.3 Qt GUI and Bokeh GUI templates

The same read-the-field and two-branch logic was added to python_qt_gui/flow_graph_qt_gui.py.mako and python_bokeh_gui/flow_graph_bokeh_gui.py.mako. These two templates were modified but their generated output has not yet been produced and run. Table 2 lists all four files.

Table 2: Files changed.

File	Change	Tested
grc/blocks/parameter.block.yml	Required field and documentation	Yes
grc/workflows/python_nogui/flow_graph_nogui.py.mako	Required or default argument	Yes
grc/workflows/python_qt_gui/flow_graph_qt_gui.py.mako	Same logic, Qt GUI	Not yet
grc/workflows/python_bokeh_gui/flow_graph_bokeh_gui.py.mako	Same logic, Bokeh GUI	Not yet

4 Testing and verification

4.1 Test setup

I tested with a small flowgraph containing two Parameter blocks, so the required and optional behaviors could be checked separately (Table 3). The flowgraph was a temporary file, /tmp/test_required_parameter.grc, and the generated script was written to /tmp/grc-required-output/test_required_parameter.py. Neither file is part of the repository.

Table 3: Test flowgraph parameters.

Parameter	Type	Configured value	Required setting
required_count	intx	10	True
optional_count	intx	20	False

Version note: the GRC compiler command I first used reported GNU Radio 3.10.12.0 and used installed blocks, which would not include my changes. To test the modified code, I loaded the source-tree workflow directory and the block definitions from grc/blocks through the GRC Platform API. Generation reported "Loading" and "Generating" lines and a warning that the flowgraph has no audio or RF hardware blocks, which is expected for an argument-parsing test.

4.2 Generated code

```
parser.add_argument(
    "--required-count", dest="required_count", type=intx, required=True,
    help="Set Required count")

parser.add_argument(
    "--optional-count", dest="optional_count", type=intx, default=20,
    help="Set Optional count [default=%(default)r]")
```

4.3 Behavior of the generated script

```
$ python3 test_required_parameter.py
usage: test_required_parameter.py [-h] --required-count REQUIRED_COUNT
                                  [--optional-count OPTIONAL_COUNT]
test_required_parameter.py: error: the following arguments are required: --required-count
(exit code 2)

$ python3 test_required_parameter.py --required-count 5
Press Enter to quit:

$ python3 test_required_parameter.py --help
  --required-count REQUIRED_COUNT   Set Required count
  --optional-count OPTIONAL_COUNT   Set Optional count [default=20]
```

The parser rejected the run without --required-count and accepted it with the argument. The help output shows the required argument without brackets and the optional one in brackets with its default. In the second run the script got past argument parsing and reached its own

prompt; that run checks parsing only, not flowgraph runtime behavior. The usage line above is wrapped for the page.

4.4 Results

Table 4: Verification results.

Check	Result	Evidence
Required argument omitted	Passed	Parser error, exit code 2
Required argument supplied	Passed	Parsing accepted the value
Optional default	Passed	default=20 in code and help text
Help message	Passed	Required and optional shown correctly
Python syntax	Passed	py_compile finished without error
Patch whitespace	Passed	git diff --check printed nothing
Final repository state	Passed	Clean tree, expected commit present
Qt GUI and Bokeh GUI output	Not run	Templates modified only
Full GRC test suite	Not run	pytest not installed in the environment
Older-flowgraph regression test	Not run	Assessed by code reading only (Section 4.5)

4.5 Backward compatibility

Block.import_data() in grc/core/blocks/block.py skips saved parameter keys that the current block definition does not know:

```
for key, value in parameters.items():
    try:
        self.params[key].set_value(value)
    except KeyError:
        continue
```

An older flowgraph has no required key, so the new field should keep its declared default, False, and the parameter stays optional. This comes from reading the import code. I have not run a test that loads an older flowgraph without the field.

4.6 Limits of this testing

These are targeted functional tests of the no-GUI workflow. They do not replace the project test suite. python3 -m pytest grc/tests failed with "No module named pytest", so the suite was not run. The Qt GUI and Bokeh GUI templates are untested.

5 Version control and pull request

The work was done on the branch grc-required-parameters. The first commit was 031c8ac51. I then amended it with git commit --amend --no-edit --signoff to add the Developer Certificate of Origin sign-off, which produced f7afafee6ccdba2ea8bd281dc6063b2f0744f136. The repository

has two remotes: origin is my fork (github.com/MuchhalAkshita/gnuradio) and upstream is gnuradio/gnuradio.

```
git commit -m "grc: add required option to parameter block"
git commit --amend --no-edit --signoff
git push -u origin grc-required-parameters
```

The first push failed with "Could not resolve host: github.com", a temporary network problem; a later push succeeded. I then opened pull request #8233 against gnuradio/gnuradio under the title "GRC: Add required option to parameter block". The description says "Closes #5096" and states the testing limitations listed above.

5.1 Maintainer and community response

The pull request is submitted. At the time of writing I have not checked reviewer comments, CI results or merge status, so this report makes no claim about them. I will update this section from the pull request page and record any requested changes.

6 Challenges

- Source tree or installed code. The regular compiler reported 3.10.12.0 and used installed blocks. Testing the change meant loading the source workflow and block directories explicitly.
- No pytest. The GRC tests could not be run, so I used a generated flowgraph for targeted checks.
- Authentication. The terminal shows nothing while a password is typed, and GitHub over HTTPS needs a personal access token rather than the account password.
- Network failure. The first push failed on name resolution and a retry worked without recreating the commit.

7 SOTA positioning

Table 5 repeats the comparison from M1. None of the reviewed works measure communication and radar performance together when both functions share one frame-timing and channel estimate on the same OFDM waveform in GNU Radio, and that remains the gap this project targets.

Table 5: Related work and the proposed project (from M1).

Work	OFDM	GNU Radio impl.	Sync/CFO robustness	Radar range / channel est.	Joint comm + radar eval.
Sturm & Wiesbeck (2011)	Yes	No	No	Yes	No
Liu et al. (2020)	Survey	No	No	Survey	No
Liu et al. (2022)	Survey	No	No	Survey	No
Dai et al. (2025)	Yes	No	No	Yes	Partial
Vaezi et al. (2026)	Partial	No	No	Partial	Partial
Qaisar et al. (2026)	No	No	No	No	No
Doly et al. (2023)	Yes	Yes	No	Yes	Partial
KIT-CEL, gr-radar	Yes	Yes	No	Yes	No
Proposed project	Yes	Yes	Yes	Yes	Yes

Where the M2 contribution sits: it changes GRC code generation, so it does not close the ISAC gap in Table 5. It is a first contribution to the GNU Radio repository, and it follows the process (branch, tests, DCO sign-off, pull request linked to an issue) that the gr-digital and gr-radar contribution planned for M3 will follow.

8 Next planned contributions

8.1 Pull request #8233

- Check CI results and read reviewer comments.
- Generate Qt GUI and Bokeh GUI test flowgraphs and repeat the required, optional and help checks.
- Install pytest and run the relevant GRC tests.
- Add a regression test that loads an older flowgraph without the required field.
- Push any requested changes to the same branch.

8.2 Remaining milestones (from the M1 timeline)

Table 6: Plan for M3 to M5.

Stage	Dates (2026)	Planned work
M3	Oct 12 to Oct 25	Implement the ISAC contribution in a GNU Radio flowgraph; unit and integration tests under the contribution guidelines.
M4	Oct 26 to Nov 8	Evaluate synchronization error, channel-estimation error, BER and range error across SNR and delay; compare with current behavior and Doly et al.
M5	Nov 9 to Nov 30	Result analysis, documentation, repository cleanup, pull request submission, final report and video.

References

1. C. Sturm and W. Wiesbeck, "Waveform design and signal processing aspects for fusion of wireless communications and radar sensing," Proceedings of the IEEE, vol. 99, no. 7, pp. 1236-1259, 2011.

2. F. Liu, C. Masouros, A. P. Petropulu, H. Griffiths, and L. Hanzo, "Joint radar and communication design: Applications, state-of-the-art, and the road ahead," IEEE Trans. Commun., vol. 68, no. 6, pp. 3834-3862, 2020.

3. F. Liu et al., "Integrated sensing and communications: Toward dual-functional wireless networks for 6G and beyond," IEEE J. Sel. Areas Commun., vol. 40, no. 6, pp. 1728-1767, 2022.

4. S. A. Doly, A. Chiriyath, W. Standage-Beier, and D. W. Bliss, "GNU Radio realization of Waveform Co-design for Joint Radar-Communications system using SDRs," Proceedings of the GNU Radio Conference, vol. 8, no. 1, 2023.

5. Communications Engineering Lab, Karlsruhe Institute of Technology, "gr-radar," GitHub, https://github.com/kit-cel/gr-radar

6. Q. Dai et al., "A Tutorial on MIMO-OFDM ISAC: From Far-Field to Near-Field," arXiv:2504.19091, 2025.

7. M. Vaezi, G. Amarasuriya Aruma Baduge, E. Ollila, and S. A. Vorobyov, "A Tutorial on AI-Empowered Integrated Sensing and Communications," IEEE Communications Surveys & Tutorials, vol. 28, 2026.

8. M. U. F. Qaisar et al., "The Role of ISAC in 6G Networks: Enabling Next-Generation Wireless Systems," IEEE Trans. Network Science and Engineering, vol. 13, 2026.

9. GNU Radio pull request #8233, https://github.com/gnuradio/gnuradio/pull/8233, and issue #5096, https://github.com/gnuradio/gnuradio/issues/5096.